

Multi-Fidelity Gaussian Process Surrogate: Structure and Training

A walkthrough of the AR1 (Kennedy-O'Hagan) covariance matrix, a concrete numerical example, and how the model is trained.

1. The AR1 Model

The AR1 (autoregressive, order 1) multi-fidelity model relates a low-fidelity (LF) simulator to a high-fidelity (HF) one via:

$$y_{\text{HF}}(\mathbf{x}) = \rho \cdot y_{\text{LF}}(\mathbf{x}) + \delta(\mathbf{x})$$

where $\mathbf{x} \in \mathbb{R}^{19}$ is the vector of tuning parameters and:

$y_{\text{LF}}(\mathbf{x})$	GP fitted primarily to LF (ne32) training data.
ρ	Learned scalar: how much the LF model scales to match HF.
$\delta(\mathbf{x})$	Discrepancy GP: HF-specific structure not captured by LF. Uncorrelated with y_{LF} by construction.

2. Fidelity Index — Augmented Input

GPpy/emukit internally appends a fidelity column to every training point so the kernel can distinguish LF from HF observations:

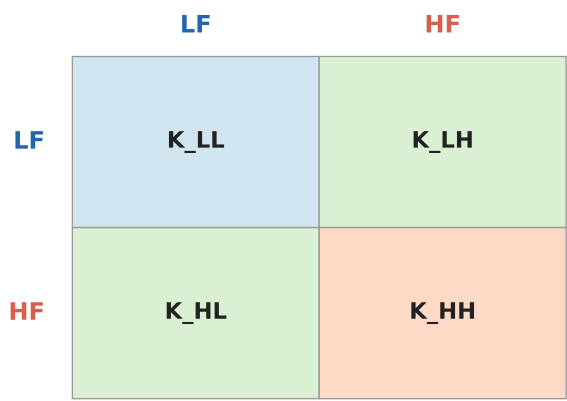
$$\tilde{\mathbf{x}}_{\text{LF}} = [x_1, \dots, x_{19}, 0] \quad \tilde{\mathbf{x}}_{\text{HF}} = [x_1, \dots, x_{19}, 1]$$

The kernel operates on this 20-dimensional augmented space and uses the last column only to decide which sub-kernel applies.

3. The Joint Covariance Matrix

For n_{LF} low-fidelity and n_{HF} high-fidelity training points the full kernel matrix $\mathbf{K}$ has size $(n_{\text{LF}} + n_{\text{HF}}) \times (n_{\text{LF}} + n_{\text{HF}})$ and a 2×2 block structure:

$\mathbf{K}$ = (block diagram below)



Block Formulas

$\mathbf{K}_{\text{LL}}$: $[\mathbf{K}_{\text{LL}}]_{ij} = k_0(\mathbf{x}_i^{\text{L}}, \mathbf{x}_j^{\text{L}})$

LF-LF: only the base kernel k_0 .

$\mathbf{K}_{\text{HH}}$: $[\mathbf{K}_{\text{HH}}]_{ij} = \rho^2 k_0(\mathbf{x}_i^{\text{H}}, \mathbf{x}_j^{\text{H}}) + k_1(\mathbf{x}_i^{\text{H}}, \mathbf{x}_j^{\text{H}})$

HF-HF: scaled base kernel plus discrepancy k_1 .

$\mathbf{K}_{\text{LH}}$: $[\mathbf{K}_{\text{LH}}]_{ij} = \rho k_0(\mathbf{x}_i^{\text{L}}, \mathbf{x}_j^{\text{H}})$

Cross block: only $\rho \cdot k_0$. No k_1 — discrepancy is uncorrelated with LF. $\mathbf{K}_{\text{HL}} = \mathbf{K}_{\text{LH}}^{\text{T}}$.

The RBF Sub-kernels (ARD)

Both k_0 and k_1 are Radial Basis Function kernels with Automatic Relevance Determination (ARD) — one lengthscale per input parameter:

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\frac{1}{2} \sum_{i=1}^{19} \frac{(x_i - x'_i)^2}{\ell_i^2}\right)$$

A large ℓ_i means the output is insensitive to parameter i ;
a small ℓ_i means the output changes rapidly with i .

4. Numerical Example — Setup

To keep the numbers readable we use 1 input parameter, 2 LF points, and 2 HF points. The structure is identical for 19 parameters and 256 points — just larger.

Training points

Point	Fidelity	Fidelity index
$x_{L1} = 0.2$	LF	0
$x_{L2} = 0.8$	LF	0
$x_{H1} = 0.3$	HF	1
$x_{H2} = 0.7$	HF	1

Kernel parameters chosen for this example

k_0 (base RBF)	$\sigma_0^2 = 1.0, \quad \ell_0 = 0.5$
k_1 (discrepancy RBF)	$\sigma_1^2 = 0.5, \quad \ell_1 = 0.3$
ρ (AR1 scale)	$\rho = 0.9 \Rightarrow \rho^2 = 0.81$

Base kernel evaluations $k_0(x, x')$

Using $k_0(x, x') = \exp(-(x - x')^2/2 \cdot 0.5^2) = \exp(-2(x - x')^2)$:

	L1=0.2	L2=0.8	H1=0.3	H2=0.7
L1=0.2	1.00	0.49	0.98	0.61
L2=0.8	0.49	1.00	0.61	0.98
H1=0.3	0.98	0.61	1.00	0.73
H2=0.7	0.61	0.98	0.73	1.00

Discrepancy kernel at HF points only: $k_1(H1, H1) = 0.50, \quad k_1(H1, H2) = 0.21, \quad k_1(H2, H2) = 0.50$

5. The Full 4×4 Joint Covariance Matrix

Assembling the blocks using the kernel values from page 3 ($\rho = 0.9$, $\rho^2 = 0.81$):

	[L1]	[L2]	[H1]	[H2]
[L1]	1.00	0.49	0.88	0.55
[L2]	0.49	1.00	0.55	0.88
[H1]	0.88	0.55	1.31	0.80
[H2]	0.55	0.88	0.80	1.31



K_LL: LF-LF: $k_0(L_i, L_j)$



K_HH: HF-HF: $\rho^2 \cdot k_0(H_i, H_j) + k_1(H_i, H_j)$ [diagonal = $0.81 \times 1.00 + 0.50 = 1.31$]



K_LH: Cross: $\rho \cdot k_0(L_i, H_j)$ [e.g. $0.9 \times 0.98 = 0.88$]

Noise: MixedNoise likelihood

GPy adds a separate noise variance to each fidelity's diagonal entries:

$$\mathbf{K}_{\text{noisy}} = \mathbf{K} + \text{diag}(\sigma_{LF}^2, \dots, \sigma_{LF}^2, \sigma_{HF}^2, \dots, \sigma_{HF}^2)$$

← n_{LF} entries →

← n_{HF} entries →

Both σ_{LF}^2 and σ_{HF}^2 are free hyperparameters learned during training.

A larger σ_{HF}^2 means the model doesn't try to interpolate every HF point exactly.

6. Training: Marginal Likelihood Maximisation

All hyperparameters are found by maximising the log marginal likelihood — the probability of observing all training outputs given the inputs and the model:

$$\log p(\mathbf{y} \mid \mathbf{X}, \theta) = -\frac{1}{2} \mathbf{y}^T \mathbf{K}_{\text{noisy}}^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_{\text{noisy}}| - \frac{n}{2} \log 2\pi$$

Here $\mathbf{y}$ stacks all training outputs (LF and HF), $\mathbf{X}$ stacks all augmented inputs, and θ is the set of hyperparameters listed below. The first term rewards fitting the data; the second penalises overly complex models (Bayesian Occam's razor). GPy maximises this using gradient ascent (L-BFGS), computing $\partial \log p / \partial \theta$ analytically via the Cholesky decomposition of $\mathbf{K}_{\text{noisy}}$.

Hyperparameters per AR1 model (43 total)

Parameter	Count	Role
$\ell_{0,1}, \dots, \ell_{0,19}$	19	Base kernel k_0 ARD lengthscales — one per tuning parameter
σ_0^2	1	Base kernel k_0 variance (overall amplitude)
$\ell_{1,1}, \dots, \ell_{1,19}$	19	Discrepancy kernel k_1 ARD lengthscales
σ_1^2	1	Discrepancy kernel k_1 variance
ρ	1	AR1 scaling between fidelities
$\sigma_{LF}^2, \sigma_{HF}^2$	2	Noise variances for LF and HF observations
	43	Total per AR1 model

Objective: Joint, not HF-only

The marginal likelihood sums over all $n_{LF} + n_{HF}$ observations. It is not weighted toward HF accuracy. HF surrogate quality emerges because the model must explain the HF observations, and k_1 and ρ are the only parameters that can absorb structure unique to HF once k_0 is constrained by the more numerous LF points.

7. Inference: Predicting at a New Parameter Set

Once training is done ($\mathbf{K}_{\text{noisy}}$ is factorised), predicting at a new parameter set $\mathbf{x}^*$ requires only a matrix–vector multiply. The GP posterior gives both a mean prediction and an uncertainty.

Predictive mean and variance

$$\mu^* = \mathbf{k}_*^T \mathbf{K}_{\text{noisy}}^{-1} \mathbf{y}$$

$$\sigma^{2*} = k_{**} - \mathbf{k}_*^T \mathbf{K}_{\text{noisy}}^{-1} \mathbf{k}_*$$

where $\mathbf{y}$ is the stacked vector of all training outputs (LF then HF), $\mathbf{k}_*$ is the $(n_{LF} + n_{HF})$ -length cross-covariance between $\mathbf{x}^*$ (at HF fidelity) and every training point, and k_{**} is the prior variance at $\mathbf{x}^*$.

What is in $\mathbf{k}_*$?

Because we predict at fidelity 1 (HF), the cross-covariance entries are read from the HF row of the block kernel:

LF entries: $[\mathbf{k}_*]_i = \rho k_0(\mathbf{x}^*, \mathbf{x}_i^L)$

one entry per LF training point ($i = 1, \dots, n_{LF}$)

HF entries: $[\mathbf{k}_*]_j = \rho^2 k_0(\mathbf{x}^*, \mathbf{x}_j^H) + k_1(\mathbf{x}^*, \mathbf{x}_j^H)$

one entry per HF training point ($j = 1, \dots, n_{HF}$)

Prior var.: $k_{**} = \rho^2 \sigma_0^2 + \sigma_1^2$

the HF self-covariance (no noise added here)

Key insight: LF data contributes to HF predictions

Even when predicting at fidelity 1 (HF), the LF training points appear in $\mathbf{k}_*$ (via ρk_0) and in $\mathbf{K}_{\text{noisy}}^{-1}$. The 175 LF runs therefore inform every HF prediction — not just as extra noise, but as signal that partially determines where μ^* lands. The more LF data, and the closer ρ is to 1, the more it helps.

Computational cost

Training (once): $O(n^3)$ — Cholesky of $\mathbf{K}_{\text{noisy}}$ ($n = 256$).
Done once; the triangular factor is stored.

Prediction per point: $O(n^2)$ — evaluate $\mathbf{k}_*$ (n kernel calls) then two triangular solves using the stored Cholesky.

Prediction per batch: $O(m \cdot n)$ — for m test points, dominated by computing the $m \times n$ matrix $\mathbf{K}_*$.

Using the uncertainty

σ^{2*} is large where training data is sparse — the model is honest about what it doesn't know. The optimizer uses only μ^* (the mean) to evaluate candidate parameter sets. The uncertainty could additionally be used for Bayesian optimisation (acquisition functions such as Expected Improvement), which would trade off exploitation vs exploration of the parameter space.

8. Worked Inference Example — Predict at $x^* = 0.5$

Same toy setup: 2 LF points (0.2, 0.8) + 2 HF points (0.3, 0.7). $y_{\text{true}}(x) = x^2$.

LF outputs = ρx^2 , HF outputs = x^2 . Noise: $\sigma_{LF}^2 = 0.10$, $\sigma_{HF}^2 = 0.05$.

Stacked output vector: $\mathbf{y} = [0.036, 0.576, 0.090, 0.490]^T$

Step 1 — Cross-covariance vector $\mathbf{k}_*$ (4 entries, one per training point)

- L1:

$\rho k_0(0.5, 0.2) = 0.9 \times e^{-0.18}$

= 0.752
- L2:

$\rho k_0(0.5, 0.8) = 0.9 \times e^{-0.18}$

= 0.752 (equal to L1 — equidistant)
- H1:

$\rho^2 k_0(0.5, 0.3) + k_1(0.5, 0.3) = 0.81 \times 0.923 + 0.400$

= 1.148
- H2:

$\rho^2 k_0(0.5, 0.7) + k_1(0.5, 0.7)$

= 1.148 (equal to H1 — equidistant)

HF entries (1.148) are larger than LF entries (0.752) because k_1 adds to $\rho^2 k_0$.

Step 2 — Prior HF variance k_{**}

$k_{**} = \rho^2 \sigma_0^2 + \sigma_1^2 = 0.81 \times 1.0 + 0.5 = 1.31$

(before seeing any data, variance = 1.31)

Step 3 — Weights $\alpha = \mathbf{K}_{\text{noisy}}^{-1} \mathbf{y}$ (solved via Cholesky)

L1	L2	H1	H2
-0.188	+0.543	-0.119	+0.153

α is not symmetric: y increases with x^2 so L2 and H2 pull harder than L1 and H1.

Step 4 — Predictive mean $\mu^* = \mathbf{k}_*^T \alpha$

$\mu^* = (0.752)(-0.188) + (0.752)(+0.543) + (1.148)(-0.119) + (1.148)(+0.153)$

= -0.141 + 0.409 + -0.137 + 0.175

= 0.306 (true value $x^{*2} = 0.25$, GP error = 0.056)

Step 5 — Predictive std $\sigma^* = \sqrt{k_{**} - \mathbf{k}_*^T \mathbf{K}_{\text{noisy}}^{-1} \mathbf{k}_*}$

$\sigma^{2*} = 1.31 - 1.224 = 0.086 \implies \sigma^* = 0.293$

The prior variance 1.31 shrinks by 1.224 because $x^*=0.5$ lies between H1=0.3 and H2=0.7.
sigma* = 0.293 is small — the model is confident here.
Far from all training points the variance would stay close to the prior 1.31.

9. Real Problem Scale and Single vs Multi-Fidelity

Scale in this application

- Training data:

$n_{LF} = 175 \text{ ne32 runs} + n_{HF} = 81 \text{ ne256 runs} \Rightarrow 256 \times 256 \text{ matrix per AR1 model}$

- Parameters:

$p = 19 \text{ tuning parameters} \Rightarrow 19 \text{ ARD lengthscales in each of } k_0, k_1$

- Number of AR1 models:

$66 \text{ (one per ZRG feature} \times \text{variable: 22 features} \times \text{3 variables)}$

- Hyperparameters (total):

$43 \times 66 = 2,838 \text{ learned independently across all models}$

Single-fidelity vs Multi-fidelity comparison

	Single-fidelity (SF)	Multi-fidelity (MF)
Framework	GPflow / ESEm	GPy / emukit
Number of models	1 joint GPR	66 AR1 models
Fidelity	HF only (ne256, 81 pts)	LF + HF (175 + 81 pts)
Hyperparameters	21 (shared across all 66 outputs)	43 per model (2,838 total, independent)
Kernel per output	Identical k for all 66	Own k_0, k_1, ρ per output
Regularisation	Strong (21 params fit from 81×66 data)	Weaker (43 params per model, 256 points)
Discrepancy	None — single fidelity	$\delta(\mathbf{x})$ captures ne256 vs ne32 difference

Interpreting the learned hyperparameters

$\rho \approx 1$ LF and HF models agree; the AR1 correction is small.

$\rho \ll 1$ or $\rho \gg 1$ LF is a poor proxy for HF; the scaling is large.

$\sigma_1^2 \gg \sigma_0^2$ Discrepancy dominates — HF has structure LF cannot explain.
Reported as discrepancy fraction $= \sigma_1^2 / (\sigma_0^2 + \sigma_1^2)$.

Small $\ell_{0,i}$ or $\ell_{1,i}$ Parameter i strongly affects the output for that ZRG feature.